



ADR-011: Vanilla JavaScript SPA with design tokens (no framework, strict CSP)

TASCO Insurance × VETC Growth Platform — Architecture Decision Record

iorta TechNXT

Version 1.0 · 07 October 2026

Contents

Context and problem statement

Decision drivers

Considered options

Decision outcome

Pros and cons of the options

Document	Version	Date	Owner	Status
ADR-011	1.0	07 October 2026	iorta TechNXT Architecture	Accepted

- **Related:** [ADR-002, Security Architecture](#), section 5

Context and problem statement

The front end has three parts:

- the staff console: dashboards, lead queue, customer 360, voice console, handoffs, rules console, partners, claims, DQ, audit and users;
- the customer web app (`/app`), embedded in the VETC app or Zalo;
- the public certificate verification page (`/verify/:certNo`).

It must be fast on mid-range Android devices, accessible and bilingual (vi/en), and must carry the TASCO brand with iorta TechNXT attribution. It must also be hardened against XSS and supply-chain risk, because staff sessions can see PII.

Decision drivers

- A strict Content-Security-Policy with no inline script or style.
- No build chain, and no npm front-end dependency tree.
- A consistent look across three surfaces.

Considered options

1. **Vanilla ES modules + CSS design tokens + small shared helpers** (`public/js/shared/api.js` , `dom.js`), served by the API.
2. React/Vue/Angular with a bundler.
3. Server-side rendered templates.

Decision outcome

Chosen option: **1.**

- `public/index.html` loads one module script (`/js/console/main.js`). Styles come from `public/css/tokens.css` (colours, spacing, type scale, radii, dark mode), `components.css` and `console.css`. Third-party code is vendored and reviewed (`public/vendor/qrcode.mjs` for QR codes on certificates). No CDN scripts are loaded.
- CSP (`src/adapters/http/security.js`): `default-src 'self'; script-src 'self'; style-src 'self'; img-src 'self' data:; font-src 'self'; connect-src 'self'; object-src 'none'; base-uri 'none'; frame-ancestors 'none'; form-action 'self'`. It is sent with `X-Frame-Options: DENY`, `nosniff`, `Referrer-Policy: no-referrer`, `COOP/CORP same-origin`, and HSTS in production.
- DOM rendering goes through helpers that build elements and set `textContent`. Any use of `innerHTML` with data is prohibited by code review.
- Static caching: `.html` is `no-cache`, `/assets/*` gets 1 day, and other files get 5 min (`app.js#serveStatic`). A path-traversal guard is applied.

Consequences

- Good: a near-zero front-end supply chain and an effective CSP against injected scripts.
- Good: instant load with no hydration cost.
- Bad: more hand-written UI code and no component ecosystem. **Mitigation:** shared DOM helpers, a component CSS layer and design tokens.
- Bad / constraints:
 - `frame-ancestors 'none'` and `X-Frame-Options: DENY` **prevent embedding /app in an iframe**. If the VETC app or Zalo mini app uses a WebView with top-level navigation, this is fine. If iframe embedding is required, relax `frame-ancestors` for `/app/*` only, to the specific VETC/Zalo origins.
 - `CORS_ORIGINS` is an allow-list for **cross-origin** callers only. Same-origin requests (the SPA's own `fetch` and module loads, which still send `Origin`) are recognised by comparing the `Origin` host with the `Host` header and always allowed (`security.js#cors`). Any other origin not in the list gets `403`.
 - The access token lives in browser memory or storage, and there is no CSRF exposure because it is a Bearer header, not a cookie. Strict CSP lowers the XSS token-theft risk.

Pros and cons of the options

React and similar. A rich ecosystem. However, bundling adds hundreds of dev dependencies, and frameworks often need CSP relaxations or nonces.

SSR templates. Less JS. However, the interactive console would need more round trips, and server-side templating adds to the XSS review surface.